KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

DAG-Based Local Code Optimization Tool

Submitted as a part of the course

Compiler Design

Submitted By

Name	Enrollment No.
Smit Shah	202307020107

Program: B.Tech CSE
Semester: VI

Under the Guidance of

Dr. Bela Shrimali
Assistant Professor
Department of Computer Science & Information Technology

Karnavati University, Gandhinagar

Academic Year: 2026–27

CERTIFICATE

This is to certify that the mini project entitled **"DAG-Based local code optimization tool"** has been successfully completed by the following students of B.Tech CSE Semester VI, Karnavati University, as a part of the course **Compiler Design** during the academic year 2026–27.

Sr No.	Student Name	Enrollment No.
1	Smit Shah	202307020107

The project has been carried out under the guidance and supervision of the faculty member. The work presented in this report is the original work carried out by the students as part of their mini project.

Faculty Guide
Name: Dr. Bela Shrimali
Signature: _____
Date: _____

DECLARATION

We hereby declare that the mini project entitled **"DAG-Based Local Code Optimization Tool"** submitted as part of the course **Compiler Design** is our original work carried out under the guidance of the faculty member.

We further declare that this work has not been submitted previously to any other institution or university for the award of any degree, diploma, or academic certification.

All research papers, datasets, frameworks, APIs, libraries, websites, and other resources used during the development of the project have been appropriately acknowledged and referenced.

Sr. No.	Student Name	Enrollment No.
1	Smit Shah	202307020107

Date: 27/10/2026
Place: Gandhinagar

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty guide, Dr. Bela Shrimali, Assistant Professor, Department of Computer Science & Information Technology, for her guidance, encouragement and valuable feedback throughout this project.

I am thankful to Karnavati University and the UnitedWorld Institute of Technology (UIT) for providing the laboratory facilities and academic environment needed to complete this work. I also acknowledge the authors of the Dragon Book, whose algorithms form the foundation of this project, and the open-source communities behind Python, FastAPI, React and React Flow. Finally, I thank my family and friends for their constant support and encouragement.

ABSTRACT

Compilers improve the efficiency of generated code through optimization. Local optimization works inside a basic block, and the Directed Acyclic Graph (DAG) is the standard data structure for it. However, DAG construction and value numbering are hard to follow from static textbook diagrams. This project presents **DAGOpt**, an interactive DAG-based local code optimization tool.

The tool parses three-address code, partitions it into basic blocks using the leader rules, builds a DAG with value numbering, and applies common subexpression elimination, constant folding and propagation, algebraic simplification and dead code elimination. Optimized code is regenerated through a topological traversal of the live nodes. The compiler engine is written in Python and exposed through a FastAPI REST service; the interface is built with React and React Flow and shows the DAG, a step-by-step construction trace and a before/after comparison.

On seven textbook benchmark blocks the tool reduced the instruction count by 42.3 % on average (up to 80 %), preserved the live-out values on 200 random inputs per benchmark, passed all 7 unit tests and 8 test cases, and optimized each block in under one millisecond. The project shows that DAG-based local optimization is effective and can be made easy to understand through visualization.

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION **8**
1.1 Introduction 8
1.2 Background 8
1.3 Problem Statement 8
1.4 Motivation 9
1.5 Objectives 9
1.6 Scope of the Project 9
CHAPTER 2 – LITERATURE REVIEW **10**
2.1 Existing Systems and Research 10
2.2 Comparative Analysis 10
2.3 Research Gap 11
CHAPTER 3 – PROPOSED SYSTEM **11**
3.1 Proposed Solution 11
3.2 System Requirements 11
3.3 Functional Requirements 12
3.4 Non-Functional Requirements 12
CHAPTER 4 – SYSTEM ARCHITECTURE **13**
4.1 Overall System Architecture 13
4.2 Architecture Components 13
4.3 Workflow 14
CHAPTER 5 – COMPILER DESIGN CONCEPTS USED **15**
5.1 Three-Address Code and Basic Blocks 15
5.2 DAG and Value Numbering 15
5.3 Optimization Techniques 16
5.4 Liveness and Dead Code Elimination 16
5.5 Code Generation from the DAG 16
CHAPTER 6 – METHODOLOGY **16**
6.1 Input Processing 16
6.2 Basic Block Partitioning 17
6.3 DAG Construction 17
6.4 Applying Optimizations 17
6.5 Liveness Analysis and Dead Code Elimination 17
6.6 Code Generation 17
6.7 Visualization and Result Presentation 17
CHAPTER 7 – IMPLEMENTATION **18**
7.1 Project Structure 18
7.2 Compiler Engine Implementation 18
7.3 REST API Implementation 19
7.4 User Interface 19
CHAPTER 8 – RESULTS AND OUTPUT **20**
8.1 Test Cases 20
8.2 Sample Outputs 21
8.3 Execution Trace 23
CHAPTER 9 – PERFORMANCE EVALUATION **23**
CHAPTER 10 – CHALLENGES AND LIMITATIONS **24**
CHAPTER 11 – FUTURE SCOPE **24**
CHAPTER 12 – CONCLUSION **25**
REFERENCES **26**
APPENDIX **26**
Appendix A – Source Code 26
Appendix B – Additional Screenshots 27
Appendix C – Test Data 27

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Overall System Architecture	13
Figure 2	DAG Construction Workflow	15
Figure 3	Main User Interface	19
Figure 4	DAG Canvas with Step Scrubber	20
Figure 5	Sample Output – Common Subexpression Elimination (TC01)	21
Figure 6	Sample Output – Constant Folding and Propagation (TC03)	22
Figure 7	Sample Output – Dead Code Elimination (TC05)	22
Figure 8	Unit Test Execution Output	23
Figure 9	Theory Reference Panel	27
Figure 10	Optimization Pass Switches and Live-Out Variable Selector	27

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Literature Survey	10
Table 2	Comparison of Existing Approaches with DAGOpt	10
Table 3	Hardware Requirements	11
Table 4	Software and Technologies Used	12
Table 5	Architecture Components	13
Table 6	Optimization Techniques Implemented	16
Table 7	REST API Endpoints	19
Table 8	Test Cases	20
Table 9	Performance Evaluation	23
Table 10	Benchmark Inputs and Live-Out Sets	27

CHAPTER 1 – INTRODUCTION

1.1 Introduction

A compiler translates a source program into target code through a series of phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization and code generation. Intermediate code is commonly written as **three-address code (TAC)**, in which every instruction has at most one operator on its right-hand side, for example `t1 = a + b`. Code optimization transforms this code so that it runs faster or uses fewer resources without changing its meaning.

Optimization can be local (inside one basic block), global (across the blocks of a procedure) or interprocedural. A **basic block** is a straight-line sequence of instructions with a single entry and a single exit. Local optimization of a basic block is performed using a **Directed Acyclic Graph (DAG)** in which each node represents a value computed in the block [1]. Repeated expressions share one node, so redundancy becomes visible automatically. This project, DAGOpt, implements this classic technique and makes it interactive.

1.2 Background

In textbooks the DAG of a basic block is drawn by hand. The student scans each statement, checks whether the same operation on the same operands already exists, and moves variable labels between nodes when a variable is reassigned. This is slow and error-prone, especially with reassignments, commutative operators, array accesses and foldable constants. Production compilers such as GCC and LLVM perform these optimizations internally, but their internal graphs are not designed for learning. A tool that builds the DAG automatically and shows each construction step is useful for understanding the algorithm and for checking hand-worked examples.

1.3 Problem Statement

The objective of this project is to develop a DAG-based local code optimization tool that accepts three-address code, builds the DAG of a basic block and produces equivalent, optimized three-address code. Existing learning material shows only static, hand-drawn DAGs and gives no way to experiment with inputs or to see how each optimization changes the graph. Therefore, there is a need for a system that can automatically apply common subexpression elimination, constant folding, algebraic simplification and dead code elimination, and visualize the construction step by step.

1.4 Motivation

- **Why this problem:** DAG-based local optimization is a core Compiler Design topic that is hard to learn from static diagrams.
- **Why automation:** it removes manual errors in value numbering and in label management during variable reassignment.
- **Who benefits:** students and instructors of Compiler Design.
- **Limitations of existing approaches:** no interactivity, no step-by-step view and no option to switch individual optimizations on or off.

1.5 Objectives

1. To parse three-address code and partition it into basic blocks using the leader rules.
2. To construct the DAG of a basic block using value numbering.
3. To implement common subexpression elimination, constant folding and propagation, algebraic simplification and dead code elimination.
4. To regenerate optimized three-address code from the DAG and report optimization statistics.
5. To build an interactive web interface that visualizes the DAG construction step by step.
6. To evaluate the tool using predefined test cases and textbook benchmarks.

1.6 Scope of the Project

- **Target users:** students, teachers and learners of Compiler Design.
- **Supported input:** TAC with binary and unary operations, constants, copies, array load/store, labels, `goto`, `if … goto` and `return`.
- **Functional scope:** local optimization of one basic block at a time, with user-selectable passes and live-out variables.
- **Technologies:** Python and FastAPI (backend); React, TypeScript and React Flow (frontend).
- **Expected usage:** run locally and use through a web browser.
- **Limitations:** no global or loop optimization, no register allocation or machine-code generation, and no source-language front end (input is TAC).

CHAPTER 2 – LITERATURE REVIEW

2.1 Existing Systems and Research

Table 1 summarizes the work most relevant to DAG-based local optimization.

Table 1: Literature Survey

Sr. No.	Paper/System	Year	Approach	Key Finding	Limitation
1	Aho, Lam, Sethi, Ullman – Compilers: Principles, Techniques, and Tools [1]	2006	DAG of a basic block with value numbering	Repeated expressions share nodes, enabling CSE and DCE	Static diagrams; no interactive tool
2	J. Cocke – Global Common Subexpression Elimination [2]	1970	Data-flow information across blocks	Redundant expressions can be removed program-wide	Needs data-flow analysis; hard to follow stepwise
3	Briggs, Cooper, Simpson – Value Numbering [3]	1997	Hash-based value numbering	Redundancy found in one pass over a block	Algorithmic description only; no visualization
4	Lattner and Adve – LLVM [4]	2004	SSA-based IR with modular optimization passes	Production-grade optimization of IR	Large and complex; not built for step-by-step learning
5	Compiler Explorer (M. Godbolt) [5]	2012	Web tool showing compiler output for source code	Lets users inspect optimized output interactively	Shows final output only, not DAG construction

2.2 Comparative Analysis

Table 2: Comparison of Existing Approaches with DAGOpt

Feature	Existing Approach (Textbook / Compilers)	DAGOpt
DAG construction	Manual on paper / hidden inside compiler	Automatic
Basic block partitioning	Manual	Automatic (leader rules)
Step-by-step trace	Not available	Available, with replay controls
DAG visualization	Static drawing	Interactive graph
Enable/disable each optimization	Not available	Available
Live-out variable selection	Fixed by the problem	Selectable by the user
Before/after statistics	Not available	Available

2.3 Research Gap

Existing resources either explain the DAG technique statically [1], [3] or hide it inside large compiler frameworks [4], [5]. None lets a learner enter their own three-address code, toggle individual optimizations and watch the DAG grow instruction by instruction. DAGOpt addresses this gap with a small, readable compiler engine and an interactive visual front end.

CHAPTER 3 – PROPOSED SYSTEM

3.1 Proposed Solution

DAGOpt is a two-tier web application. The user types TAC in the editor or loads a preset, chooses which optimization passes to enable and which variables are live at block exit. The frontend sends this to the FastAPI backend, where the parser converts the text into instruction objects and the partitioner splits them into basic blocks. The DAG builder then processes the chosen block instruction by instruction using value numbering, trying constant folding, algebraic simplification and common subexpression elimination before creating any new node. A mark-sweep pass from the live-out variables flags dead nodes, and the code generator emits optimized TAC from the live nodes in topological order. The response carries the DAG, per-step snapshots, optimized code and statistics, which the interface shows as an interactive graph, a step scrubber and a comparison panel.

3.2 System Requirements

3.2.1 Hardware Requirements

Table 3: Hardware Requirements

Component	Requirement
Processor	Any modern dual-core CPU (x86-64 or ARM)
RAM	4 GB minimum (8 GB recommended)
Storage	500 MB free space (mostly frontend packages)
GPU	Not required
Internet Connection	Needed only to install dependencies and load web fonts

3.2.2 Software Requirements

The tool is built with FastAPI [6], React [7], React Flow [8] and Dagre [9]; Table 4 lists the technologies used.

Table 4: Software and Technologies Used

Software/Technology	Version/Specification
Operating System	Windows 10/11 (also runs on Linux and macOS)
Programming Languages	Python 3.10+ (backend), TypeScript 5.7 (frontend)
IDE/Environment	Any code editor (e.g., VS Code); Node.js 18+ with npm
Backend Framework	FastAPI ≥ 0.110, Uvicorn ≥ 0.28, Pydantic ≥ 2.6
Testing	pytest ≥ 8.0; built-in runner `python -m backend.test_compiler`
Frontend	React 19, Vite 6
Graph Visualization	@xyflow/react (React Flow) 12, dagre 0.8
Icons	lucide-react
Version Control	Git and GitHub

3.3 Functional Requirements

1. Accept three-address code from the user or load a preset benchmark.
2. Validate the syntax and report errors with the line number.
3. Partition the code into basic blocks and let the user select a block.
4. Build the DAG of the selected block using value numbering.
5. Apply CSE, constant folding and propagation, algebraic simplification and DCE, each switchable.
6. Let the user choose the live-out variables.
7. Generate optimized TAC with statistics (instructions saved, reduction %, CSE hits, folds, pruned nodes).
8. Visualize the DAG and replay its construction step by step.
9. Handle empty or invalid input with clear error messages.

3.4 Non-Functional Requirements

- **Accuracy:** optimized code must preserve the values of all live-out variables.
- **Reliability:** invalid input is rejected with an error message instead of a crash.
- **Performance:** a basic block is optimized in well under a second.
- **Scalability:** the REST API is stateless, so it can serve many requests independently.

- **Security:** user code is only parsed, never executed; the open CORS setting is intended for local development only.
- **Usability:** preset examples, a theory panel and a step scrubber make the tool easy to learn.
- **Maintainability:** parser, DAG builder and optimizer are separate modules with unit tests.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 Overall System Architecture

Figure 1 shows the architecture. The React frontend communicates with the FastAPI service over HTTP/JSON; the service calls the Python compiler engine, which parses, partitions, builds the DAG, eliminates dead code and generates the optimized code.

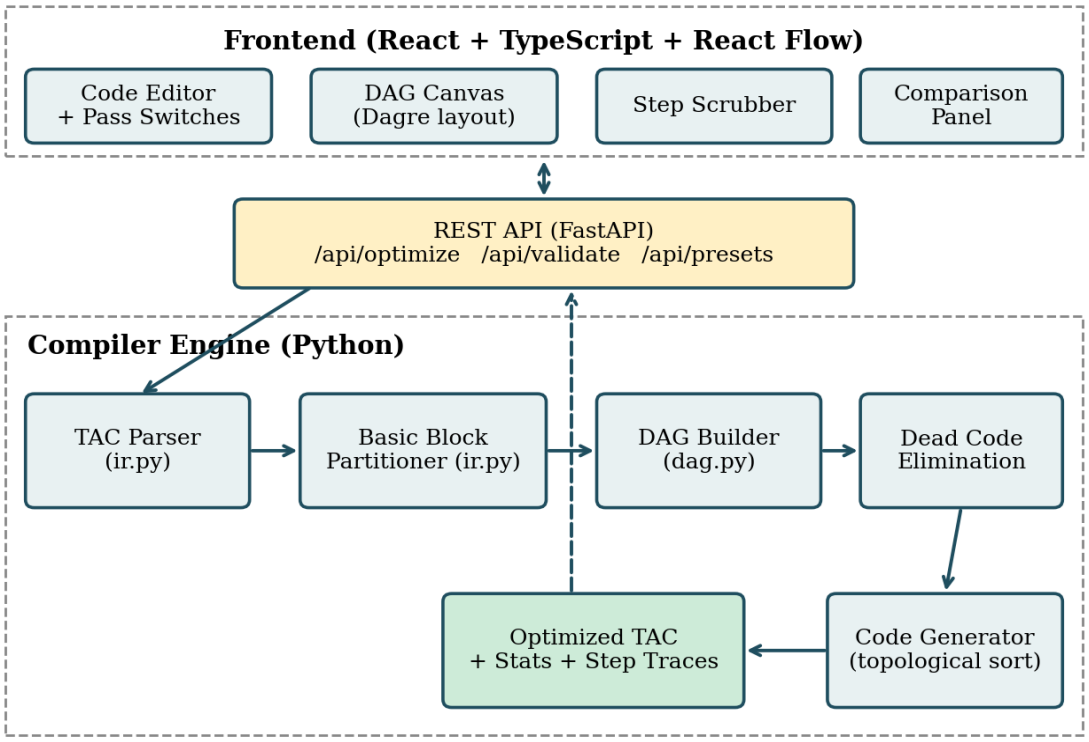


Figure 1: Overall System Architecture

4.2 Architecture Components

Table 5: Architecture Components

Component	Description
Code Editor	Line-numbered TAC input with pass switches and live-out variable selector (`CodeEditor.tsx`).
DAG Canvas	Draws the DAG with React Flow and a Dagre hierarchical layout; nodes are colour-coded as shared, folded or dead.
Step Scrubber	Play, pause and step controls to replay DAG construction instruction by instruction.
Comparison Panel	Original vs. optimized code, reduction metrics and a timeline of optimization decisions.
API Layer	FastAPI endpoints `/api/optimize`, `/api/validate`, `/api/presets`, `/api/health` (`main.py`).
TAC Parser	Converts source lines into typed `Instruction` objects and reports syntax errors (`ir.py`).
Block Partitioner	Finds leaders and splits the instruction list into basic blocks (`ir.py`).
DAG Builder	Value numbering, constant folding, algebraic simplification, CSE, array versioning and step traces (`dag.py`).
Optimizer	Live-out determination, dead code elimination, code generation, statistics (`optimizer.py`).
Presets	Seven textbook benchmark programs (`presets.py`).

4.3 Workflow

1. The user enters TAC (or loads a preset) and selects the passes and live-out variables.
2. The backend parses the text into instructions; syntax errors stop the process with a message.
3. The instructions are partitioned into basic blocks and the target block is selected.
4. For every instruction the DAG builder tries constant folding, then algebraic simplification, then a CSE lookup; otherwise it creates a new node.
5. Each decision is saved as a step trace with a snapshot of the DAG.
6. Dead nodes, i.e. nodes unreachable from live-out variables, are marked.
7. Optimized TAC is generated from the live nodes in topological order.
8. The frontend renders the DAG, the step trace, the optimized code and the statistics.

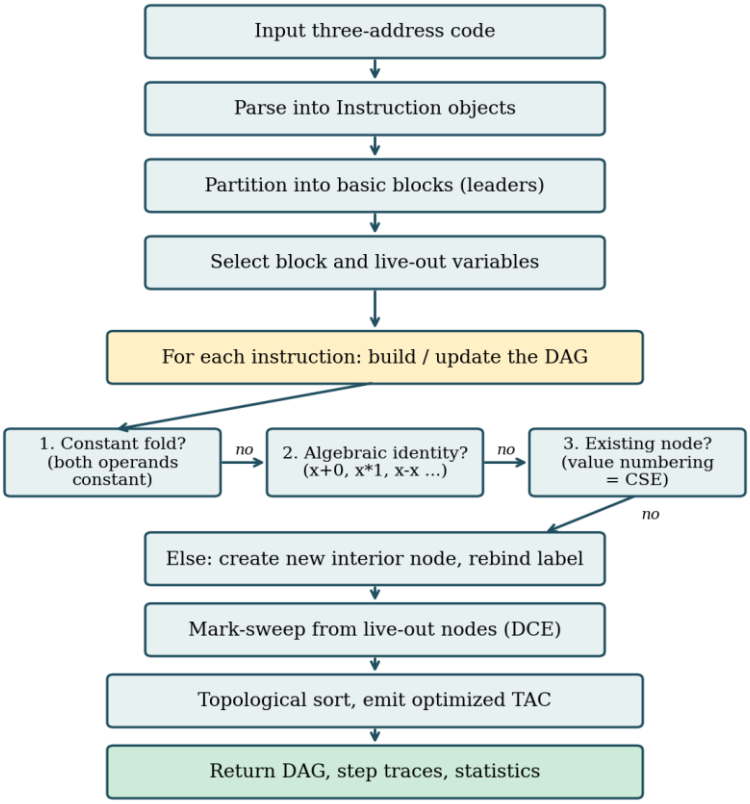


Figure 2: DAG Construction Workflow

CHAPTER 5 – COMPILER DESIGN CONCEPTS USED

5.1 Three-Address Code and Basic Blocks

A basic block is found by identifying **leaders** [1]:

- the first instruction of the program;
- any instruction that is the target of a conditional or unconditional jump;
- any instruction that immediately follows a jump or a `return`.

A block extends from a leader up to, but not including, the next leader.

5.2 DAG and Value Numbering

Leaf nodes hold the initial value of a variable or a constant; interior nodes hold an operator applied to child nodes. Each node keeps a list of variable labels. When a variable is reassigned it is removed from its old node and attached to the new one. Before creating a node for `x op y`, the builder looks up the key `(op, node(x), node(y))` in a hash table; if it exists, the node is reused [3]. For commutative operators (`+`, `*`, `&`, `|`, `^`, `==`, `!=`) both operand orders are

checked. Array reads carry a version number that is incremented on every store to that array, so a read is never reused across a store.

5.3 Optimization Techniques

Table 6: Optimization Techniques Implemented

Technique	Purpose	Example Input	Output
Common Subexpression Elimination	Reuse an expression that is already computed	`a = b + c`, `d = b + c`	`d` attached to the node of `a`
Constant Folding and Propagation	Evaluate constant expressions at compile time	`x = 10`, `y = 20`, `z = x + y`	`z = 30`
Algebraic Simplification	Apply identities such as x+0, x*1, x*0, x-x	`a = x + 0`	`a = x`
Dead Code Elimination	Remove values that never reach a live-out variable	Nodes unreachable from live-out	Pruned nodes; no code emitted

5.4 Liveness and Dead Code Elimination

The user may declare the variables that are live at the end of the block. If none are given, all non-temporary variables are treated as live (names such as `t1` or `_x` are considered temporaries unless the block contains nothing else). A mark-sweep traversal starts from nodes that carry a live-out label, plus array stores, which have side effects. All reachable children are marked live; the remaining nodes are flagged dead and shown as pruned in the graph.

5.5 Code Generation from the DAG

Live nodes are ordered topologically (children before parents) using Kahn's algorithm. For each interior node one label receives the operation; other labels bound to the same node are emitted as copy statements (`d = a`). Constant and copy statements are emitted only for live-out variables.

CHAPTER 6 – METHODOLOGY

6.1 Input Processing

The editor text is sent to the backend as a string. The parser ignores blank lines and comments (`#` or `//`) and matches each line against the supported TAC forms (binary, unary, literal, copy, array load/store, label, `goto`, `if … goto`, `return`). A line that matches none raises a syntax error carrying the line number, which the API returns to the user.

6.2 Basic Block Partitioning

Leaders are marked as described in Section 5.1 and the instruction list is cut into blocks named B1, B2, …. The user may select any block; B1 is optimized by default.

6.3 DAG Construction

For each instruction, operands are resolved to nodes: a variable already defined in the block maps to its current node, an undefined variable becomes a leaf, and a literal becomes a constant leaf. The destination variable is then bound to the resulting node.

6.4 Applying Optimizations

The decision order for a binary operation is fixed: **(A)** if both operands are constants, fold; **(B)** otherwise, if an algebraic identity applies, reuse the simplified operand or constant; **(C)** otherwise, look up the expression key for CSE; **(D)** otherwise, create a new interior node. Division or modulo by zero is never folded. Every decision is recorded as a step trace of type `CONSTANT_FOLD`, `ALGEBRAIC_SIMPLIFY`, `CSE_HIT`, `NEW_NODE` or `ARRAY_OP`.

6.5 Liveness Analysis and Dead Code Elimination

Live-out variables are taken from the user, or from the default rule in Section 5.4. Nodes not reachable from them are marked dead.

6.6 Code Generation

Live nodes are sorted topologically and optimized TAC is emitted as described in Section 5.5. Statistics are computed as reduction % = (original − optimized) / original × 100.

6.7 Visualization and Result Presentation

The JSON response contains nodes, edges, step snapshots, the variable table, optimized code, statistics and plain-language explanations. The frontend lays out the graph with Dagre, highlights the node touched by the current step, and shows original and optimized code side by side.

CHAPTER 7 – IMPLEMENTATION

7.1 Project Structure

```
DAGOpt/
├── backend/
│   ├── compiler/
│   │   ├── ir.py          # Instruction model, TAC parser, block partitioner
│   │   ├── dag.py         # DAG nodes/edges, value numbering, step traces
│   │   ├── optimizer.py   # Liveness, DCE, code generation, statistics
│   │   └── presets.py     # Seven textbook benchmark programs
│   ├── main.py            # FastAPI application and endpoints
│   ├── test_compiler.py   # Unit tests
│   └── requirements.txt
├── frontend/
│   ├── package.json, vite.config.ts, tsconfig.json
│   └── src/
│       ├── App.tsx, api.ts, types.ts, index.css
│       └── components/    # Header, CodeEditor, DagCanvas, CustomDagNode,
│                          # StepScrubber, ComparisonPanel, TheoryModal
└── README.md
```

7.2 Compiler Engine Implementation

Each DAG node stores its kind, operator, children, variable labels and flags that the interface uses for colouring:

```
@dataclass
class DagNode:
    id: str
    kind: str      # LEAF_VAR, LEAF_CONST, INTERIOR_OP, ARRAY_OP
    op: str
    children: List[str]
    labels: List[str]          # variables currently holding this value
    const_value: Optional[...] = None
    is_reused: bool = False    # CSE hit
    is_folded: bool = False    # constant folded
    is_dead: bool = False      # pruned by DCE
```

The core of common subexpression elimination is the hash-table lookup in `DAGBuilder.process_instruction`, which also tries the swapped operand order for commutative operators:

```
key = (inst.op, left_id, right_id)
alt = (inst.op, right_id, left_id) if inst.op in COMMUTATIVE_OPS else None
hit = self.expr_to_node.get(key) or (alt and self.expr_to_node.get(alt))
if hit and self.enable_cse:
    self._assign_variable_to_node(inst.dst, hit)   # reuse node, add label
    self.cse_count += 1
    return
```

Dead code elimination is a mark-sweep over the finished DAG (simplified from `DAGOptimizer._apply_dead_code_elimination`):

```
for nid, node in builder.nodes.items():                # 1. roots
    is_store = node.kind == "ARRAY_OP" and node.op == "[]="
    if is_store or any(l in live_out for l in node.labels):
        live.add(nid); queue.append(nid)
while queue:                                           # 2. propagate
    for child in builder.nodes[queue.popleft()].children:
        if child not in live:
            live.add(child); queue.append(child)
for nid, node in builder.nodes.items():                # 3. sweep
    node.is_dead = nid not in live
```

7.3 REST API Implementation

Table 7: REST API Endpoints

Endpoint	Method	Purpose
`/api/optimize`	POST	Parses the code, selects a block and returns the DAG, step traces, optimized code and statistics. Empty or invalid code returns HTTP 400.
`/api/validate`	POST	Checks the syntax and returns the detected basic blocks, or an error message.
`/api/presets`	GET	Returns the seven benchmark programs.
`/api/health`	GET	Returns the service status.

7.4 User Interface

The interface (React 19 + TypeScript) has a header with the preset selector, a theory reference and the Optimize button; a code editor with pass switches and a live-out variable selector; a DAG canvas; a step scrubber; and a comparison panel. Figures 3 and 4 show the main screens.

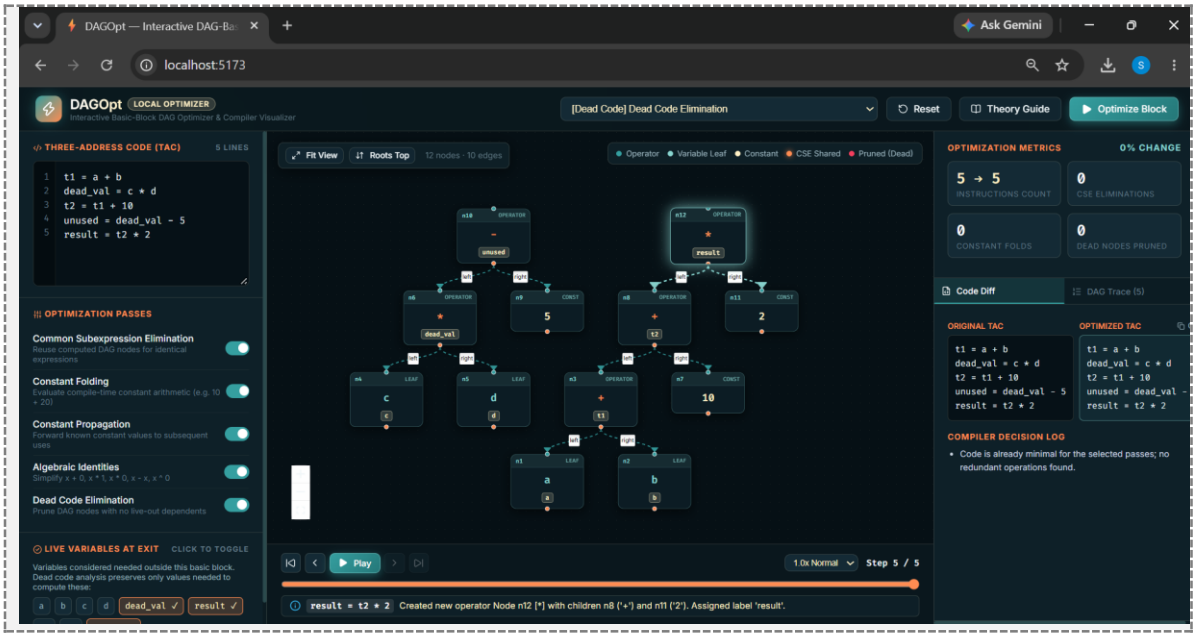


Figure 3: Main User Interface

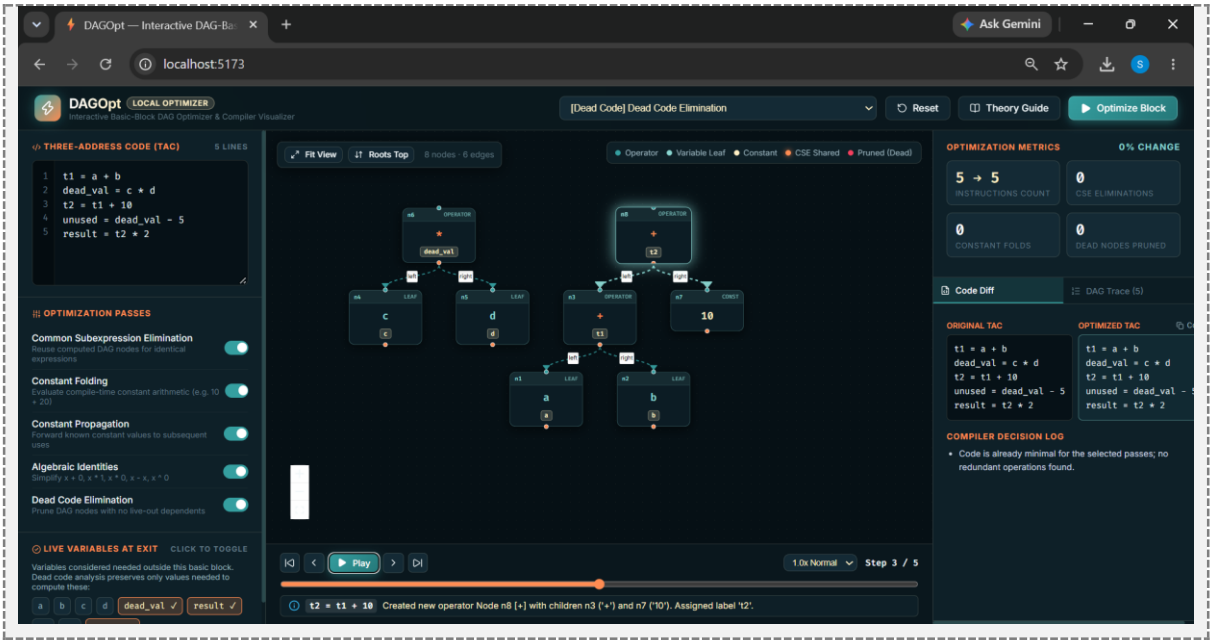


Figure 4: DAG Canvas with Step Scrubber

CHAPTER 8 – RESULTS AND OUTPUT

8.1 Test Cases

All test cases were run on the compiler engine; "live" lists the live-out variables selected. Every case produced the expected output (Table 8).

Table 8: Test Cases

ID	Input	Expected Output	Actual Output	Status
TC01	`a = b + c; d = b + c; e = a * d` (live: e)	`b + c` computed once	`a = b + c; e = a * a` (3 → 2, 1 CSE hit)	Pass
TC02	`a = b + c; d = c + b`	Commutative match; one node	`a = b + c; d = a` (1 CSE hit)	Pass
TC03	`x = 10; y = 20; z = x + y; w = z * 2; ans = w + a` (live: ans)	Constants folded to 60	`ans = 60 + a` (5 → 1, 2 folds)	Pass
TC04	`a = x + 0; b = y * 1; c = z * 0; d = w - w; result = a + b` (live: result)	Identities simplified	`result = x + y` (5 → 1, 4 simplifications)	Pass
TC05	`t1 = a + b; dead_val = c * d; t2 = t1 + 10; unused = dead_val - 5; result = t2 * 2` (live: result)	Dead statements removed	`t1 = a + b; t2 = t1 + 10; result = t2 * 2` (5 → 3)	Pass

ID	Input	Expected Output	Actual Output	Status
TC06	`t1 = 4 * i; t2 = a[t1]; t3 = 4 * i; t4 = b[t3]; t5 = t2 + t4` (live: t5)	Second `4 * i` removed	`t1 = 4 * i; t2 = a[t1]; t4 = b[t1]; t5 = t2 + t4` (5 → 4)	Pass
TC07	8-line program with `if … goto`, `goto` and two labels	Split into basic blocks	4 blocks (B1–B4), 2 instructions each	Pass
TC08	`a = = b`	Syntax error reported	"Syntax error at line 1: … is not a valid 3-Address instruction."	Pass

8.2 Sample Outputs

Test Case 1 (TC01) – Common Subexpression Elimination

Input: `a = b + c`, `d = b + c`, `e = a * d`; live-out: `e`.

Tool Action: step 1 creates node n3 [+] for `a`; step 2 finds `b + c` already computed and attaches label `d` to n3 (CSE hit); step 3 creates n4 [*] with n3 as both children.

Output: `a = b + c`, `e = a * a` (33.3 % fewer instructions).

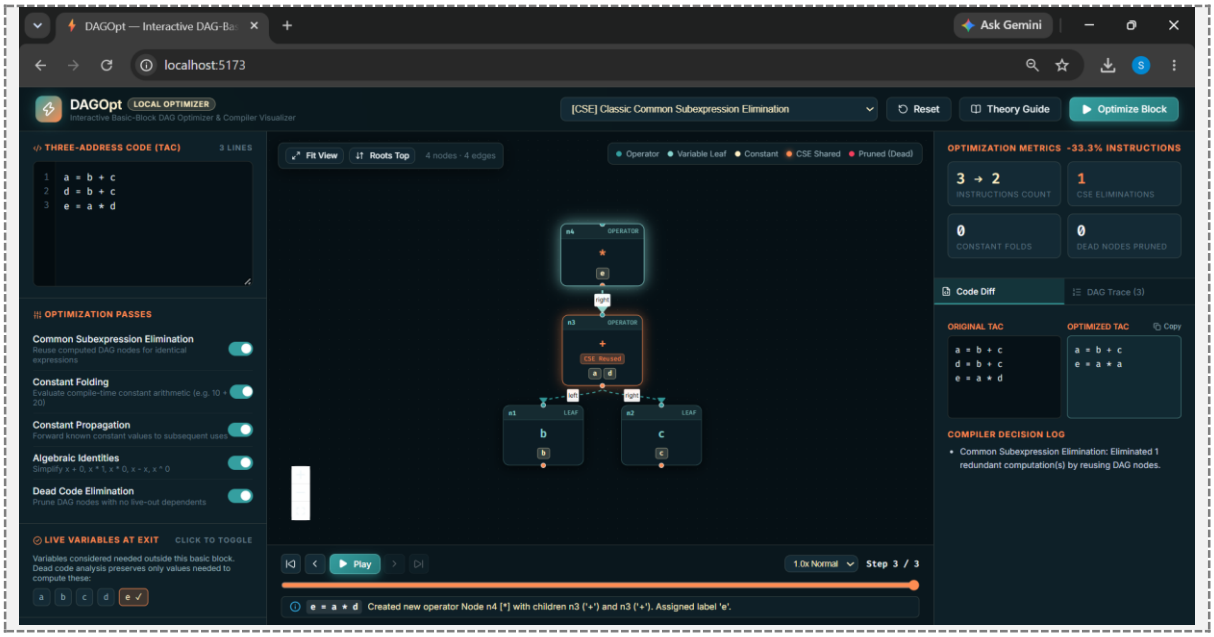


Figure 5: Sample Output – Common Subexpression Elimination (TC01)

Test Case 3 (TC03) – Constant Folding and Propagation

Input: `x = 10`, `y = 20`, `z = x + y`, `w = z * 2`, `ans = w + a`; live-out: `ans`.

Tool Action: `z` is folded to 30 and `w` to 60; the constant nodes are no longer needed and are pruned.

Output: `ans = 60 + a` (80 % fewer instructions).

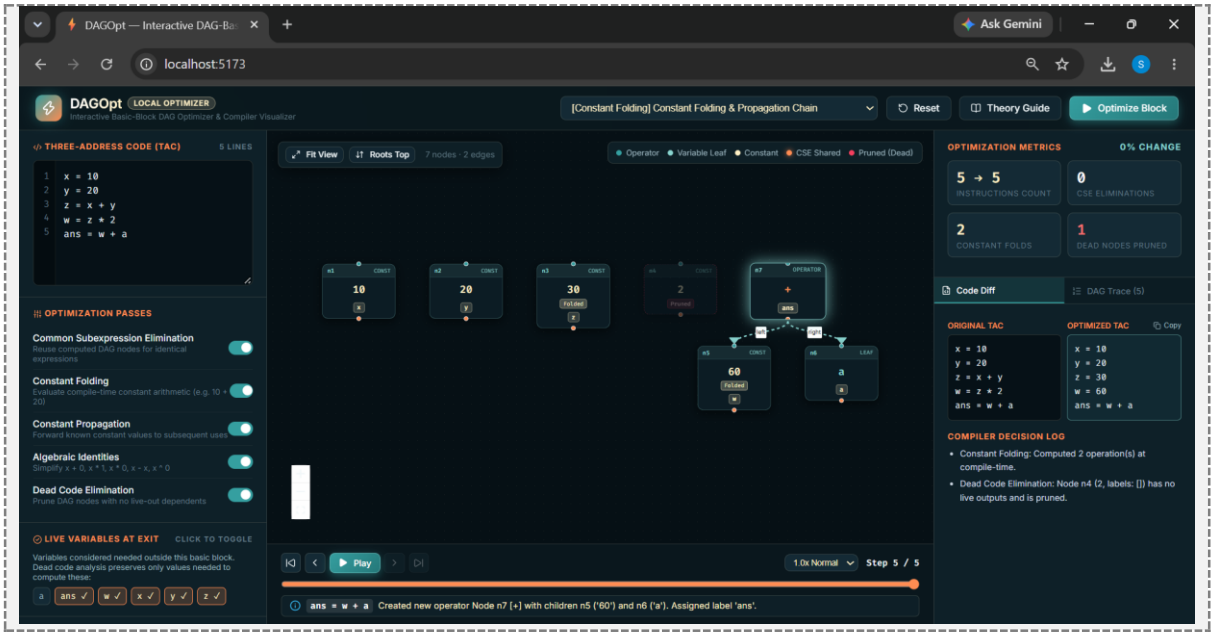


Figure 6: Sample Output – Constant Folding and Propagation (TC03)

Test Case 5 (TC05) – Dead Code Elimination

Input: `t1 = a + b`, `dead_val = c * d`, `t2 = t1 + 10`, `unused = dead_val - 5`, `result = t2 * 2`; live-out: `result`.

Tool Action: nodes of `dead_val` and `unused` are not reachable from `result` and are marked dead (shown as pruned in the graph).

Output: `t1 = a + b`, `t2 = t1 + 10`, `result = t2 * 2` (40 % fewer instructions).

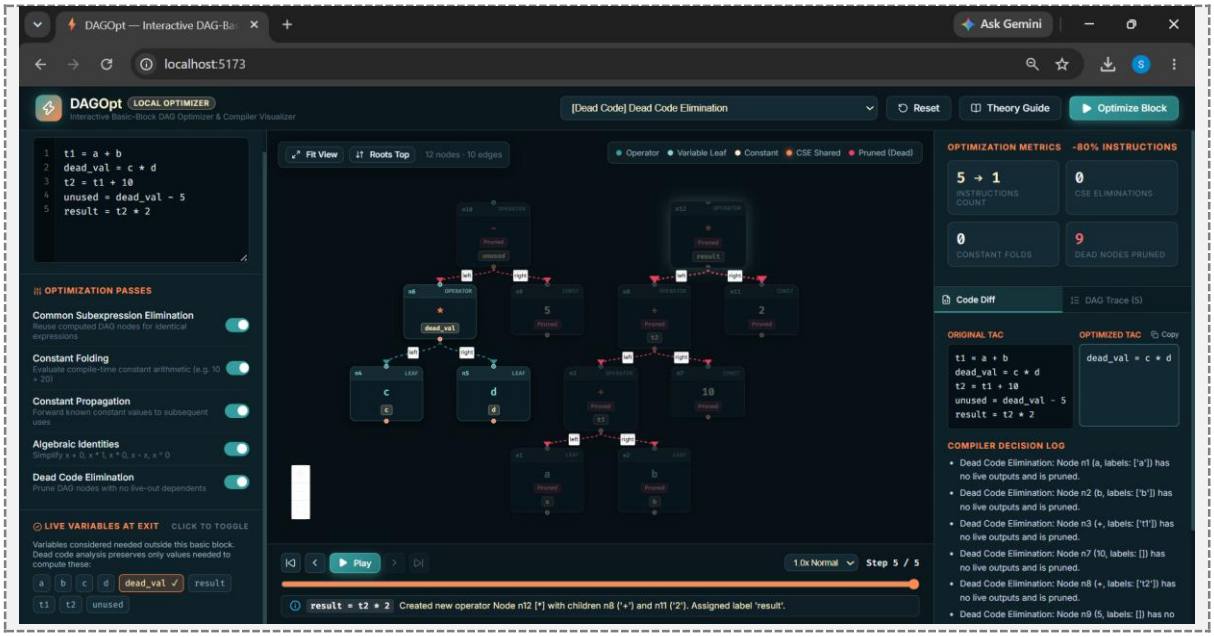


Figure 7: Sample Output – Dead Code Elimination (TC05)

The automated unit tests are run with `python -m backend.test_compiler`; Figure 8 shows the result.

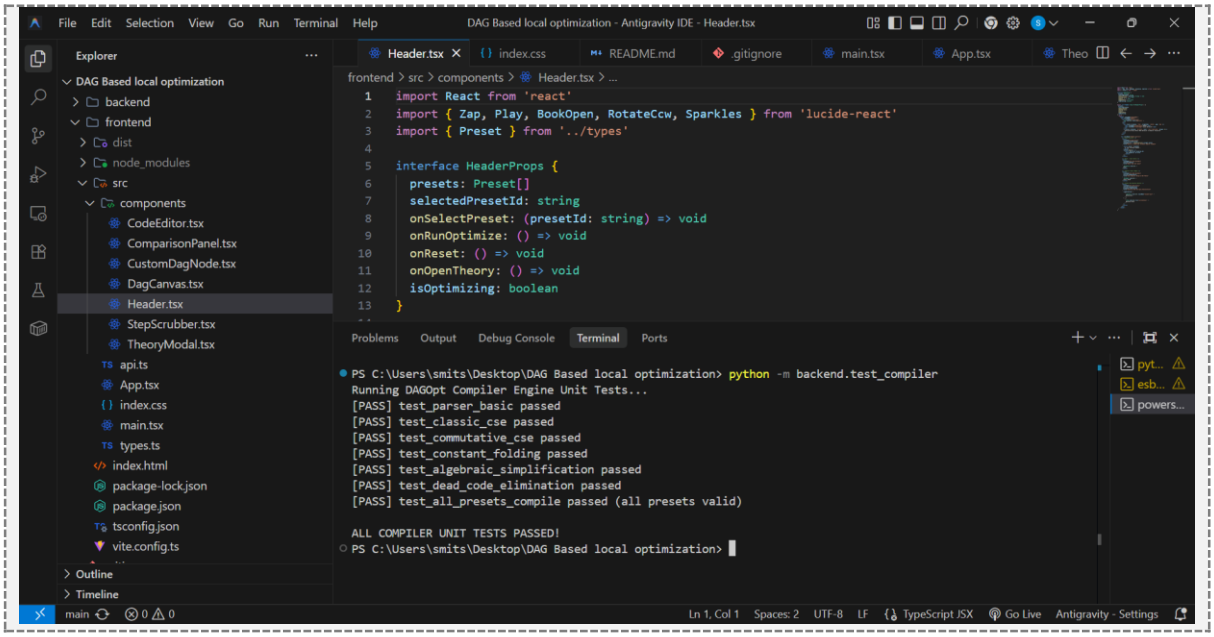


Figure 8: Unit Test Execution Output

8.3 Execution Trace

Execution trace of TC01 (values taken from the tool's step traces):

```
Input TAC         : a = b + c ; d = b + c ; e = a * d        (live-out: e)
Step 1  a = b + c : NEW_NODE  -> n3 [+](n1=b, n2=c), labels {a}
Step 2  d = b + c : CSE_HIT   -> n3 reused, labels {a, d}
Step 3  e = a * d : NEW_NODE  -> n4 [*](n3, n3), labels {e}
Dead code elim.   : all nodes reachable from live-out e (0 pruned)
Code generation   : topological order n1, n2, n3, n4
Output TAC        : a = b + c ; e = a * a
```

CHAPTER 9 – PERFORMANCE EVALUATION

The metrics in Table 9 were measured on the seven preset benchmarks with the live-out sets listed in Appendix C.

Table 9: Performance Evaluation

Metric	Evaluation Method	Result
Test Case Success Rate	Passed tests / total tests (Table 8)	8 / 8 (100 %)
Unit Test Success Rate	`python -m backend.test_compiler`	7 / 7 (100 %)
Semantic Correctness	Original and optimized TAC evaluated on 200 random inputs per benchmark; live-out values compared	7 / 7 benchmarks equivalent
Average Instruction Reduction	(Original − optimized) / original, averaged over the 7 benchmarks	42.3 % (range 0 % – 80 %)

Metric	Evaluation Method	Result
Average Optimization Time	Mean of 1000 runs per benchmark (optimizer only)	0.47 ms (0.24 – 0.86 ms)
API Verification	FastAPI test client: health, presets (7), validate, optimize, empty input	All responses as expected (empty input → HTTP 400)

Timings were measured on a single development machine and will vary with hardware. The reduction depends on the live-out set: with the default rule (all non-temporary variables live), TC01 shows 0 % reduction because `d` must still be computed. Benchmarks with no redundant work, such as variable reassignment, show 0 % reduction by design.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

- **Label management:** correctly moving variable labels between nodes on reassignment (e.g., `b = a - d` followed by `d = a - d`) required careful handling.
- **Arrays:** reads are versioned per array name; there is no alias analysis between different arrays or store-to-load forwarding.
- **Live-out dependence:** the amount of optimization depends on the chosen live-out variables; the default rule is only a heuristic.
- **Single block:** only one basic block is optimized at a time; nothing is propagated across blocks.
- **Instruction order:** code generation follows topological order, so independent statements may appear in a different order than the input.
- **Deployment:** the API allows all origins for local development and is not hardened for public deployment.

CHAPTER 11 – FUTURE SCOPE

1. Global optimization using control-flow graphs and data-flow analysis (global liveness, reaching definitions).
2. Loop optimizations such as invariant code motion and strength reduction.
3. Copy propagation and further algebraic rules.
4. Simple alias analysis for arrays.
5. Register allocation and target-code generation from the optimized DAG.

6. Optimizing all blocks at once and showing the control-flow graph.
7. Exporting DAGs and reports as images or PDF, and deploying the tool online.

CHAPTER 12 – CONCLUSION

This project addressed the difficulty of learning and verifying DAG-based local code optimization, which is usually taught with static, hand-drawn diagrams. DAGOpt parses three-address code, partitions it into basic blocks, builds the DAG using value numbering and applies common subexpression elimination, constant folding and propagation, algebraic simplification and dead code elimination. Optimized code is regenerated by a topological traversal of the live nodes.

The compiler engine was written in Python and served through FastAPI, while a React and React Flow interface shows the DAG, a replayable step trace and a before/after comparison. On the seven benchmark blocks the tool reduced the instruction count by 42.3 % on average and by up to 80 %, preserved the live-out values on random test inputs, passed all unit tests and test cases, and optimized each block in under one millisecond.

The main learning from the project was how value numbering, label rebinding and liveness interact: the quality of local optimization depends directly on which variables are live at block exit. The work can be extended to global optimization, loop optimizations and target-code generation.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson Education, 2006.

[2] J. Cocke, “Global Common Subexpression Elimination,” ACM SIGPLAN Notices, vol. 5, no. 7, 1970.

[3] P. Briggs, K. D. Cooper and L. T. Simpson, “Value Numbering,” Software: Practice and Experience, vol. 27, no. 6, 1997.

[4] C. Lattner and V. Adve, “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation,” in Proc. Int. Symp. Code Generation and Optimization (CGO), 2004.

[5] M. Godbolt, “Compiler Explorer,” 2012. [Online]. Available: https://godbolt.org

[6] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com

[7] Meta Platforms, “React Documentation.” [Online]. Available: https://react.dev

[8] xyflow, “React Flow Documentation.” [Online]. Available: https://reactflow.dev

[9] dagre, “Dagre: Directed Graph Layout for JavaScript.” [Online]. Available: https://github.com/dagrejs/dagre

APPENDIX

Appendix A – Source Code

GitHub Repository: https://github.com/ayoitssmit/DAGOpt

Deployed Link: Not deployed. The tool runs locally (backend at http://127.0.0.1:8000, frontend at http://localhost:5173).

Commands to run the project:

```
python -m pip install fastapi uvicorn pydantic
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
cd frontend && npm install && npm run dev
python -m backend.test_compiler          # unit tests
```

Appendix B – Additional Screenshots

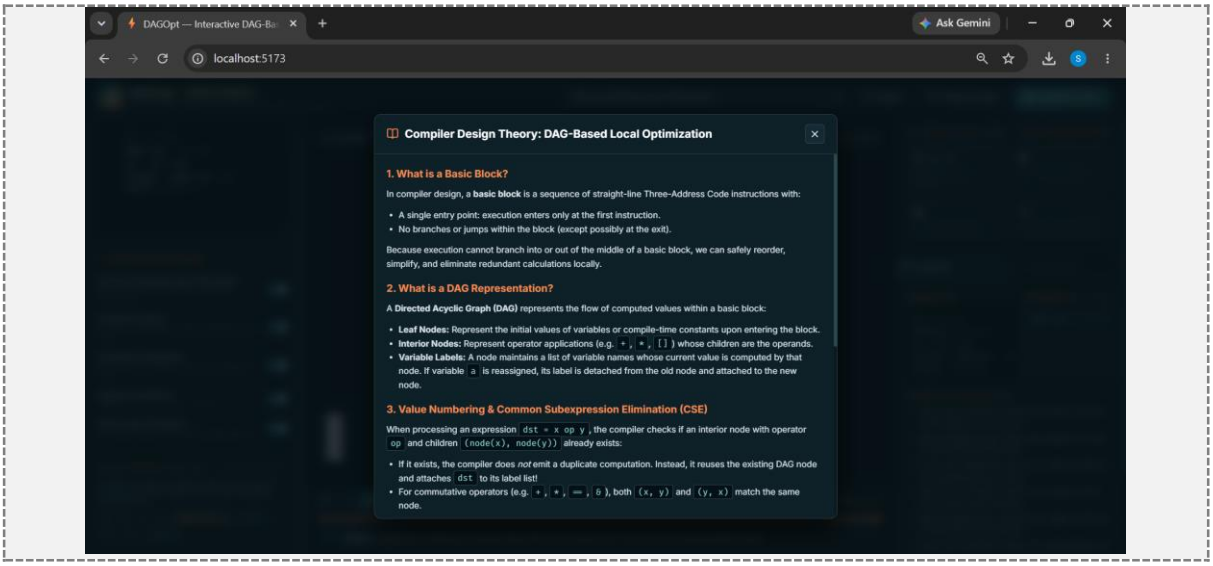


Figure 9: Theory Reference Panel

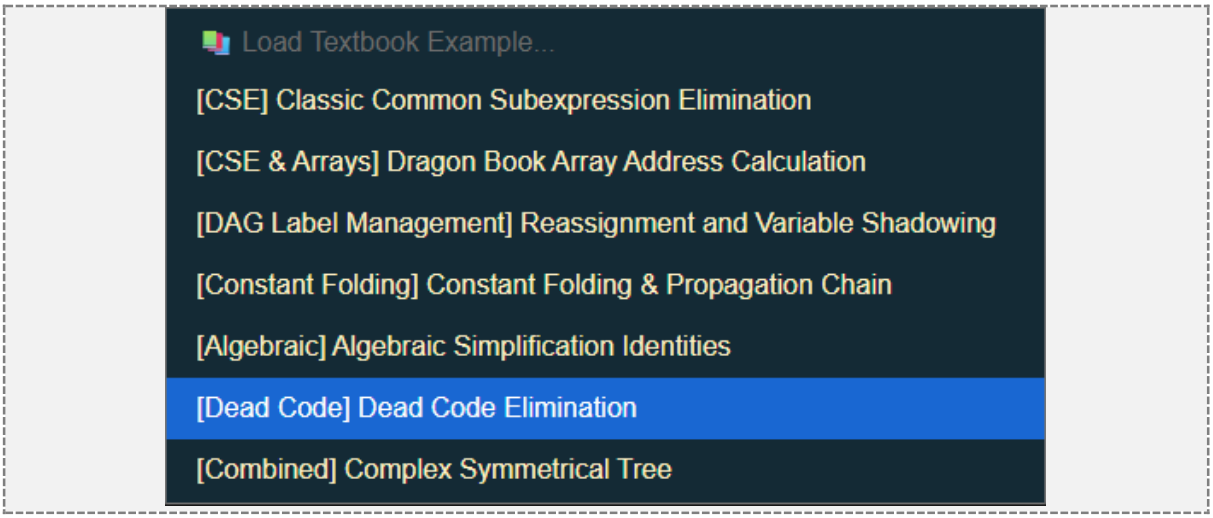


Figure 10: Optimization Pass Switches and Live-Out Variable Selector

Appendix C – Test Data

The seven preset benchmark programs used for evaluation (statements separated by semicolons):

Table 10: Benchmark Inputs and Live-Out Sets

Sr. No.	Input (TAC)	Live-out	Purpose
1	`a = b + c; d = b + c; e = a * d`	e	Classic CSE
2	`t1 = 4 * i; t2 = a[t1]; t3 = 4 * i; t4 = b[t3]; t5 = t2 + t4`	t5	Array address CSE
3	`a = b + c; b = a - d; c = b + c; d = a - d`	a, b, c, d	Reassignment and labels

Sr. No.	Input (TAC)	Live-out	Purpose
4	`x = 10; y = 20; z = x + y; w = z * 2; ans = w + a`	ans	Constant folding chain
5	`a = x + 0; b = y * 1; c = z * 0; d = w - w; result = a + b`	result	Algebraic identities
6	`t1 = a + b; dead_val = c * d; t2 = t1 + 10; unused = dead_val - 5; result = t2 * 2`	result	Dead code elimination
7	`t1 = a + b; t2 = c + d; t3 = t1 * t2; t4 = a + b; t5 = c + d; t6 = t4 * t5; res = t3 + t6`	res	Shared symmetrical subtrees